Universidade Federal de Goiás
Instituto de Informática
Compiladores e Compiladores 1
2026-1

Trabalho 1 - Implementação Compilador para a Linguagem G-V1

Prof. Thierson Couto Rosa

1 Objetivo

Este trabalho tem como objetivo a implementação de um compilador didático para a linguagem G-V1. A linguagem será estendida no trabalho T2, incluindo fuções e o tipo vetor. A linguagem G-V1 contém apenas um módulo principal. É estruturada em blocos. Cada bloco pode conter uma ou mais declarações de variáveis e uma lista de comandos. Um dos comandos pode ser outro bloco. Portanto, um mesmo nome de variável pode estar associado a variáveis de blocos distintos. As variáveis da linguagem podem estar associadas a apenas dois tipos: *car* e *int*.

2 Gramática para a linguagem *G-V1*

A seguir, é apresentada a gramática para a linguagem *G-V1*. Não será apresentada a quádrupla completa correspondente à gramática que descreve a linguagem. O conjunto V_N de símbolos não-terminais da gramática é formado pelo conjunto de símbolos nas regras de derivação que iniciam com uma letra maiúscula e que aparecem do lado esquerdo da seta em alguma produção.

O conjunto V_T de terminais da gramática é formado por todos os símbolos escritos completamente em letras maiúscula e por símbolos que aparecem entre apóstrofes.

$$
\begin{array}{lcl}
Programa & \rightarrow & DeclPrograma \\
\\
DeclPrograma & \rightarrow & \texttt{PRINCIPAL}\ Bloco \\
\\
Bloco & \rightarrow & \texttt{'\{'}ListaComando\ \texttt{'\}'} \\
 & & \mid VarSection\ \texttt{'\{'}\ ListaComando\ \texttt{'\}'} \\
\\
VarSection & \rightarrow & \texttt{'\{'}\ ListaDeclVar\ \texttt{'\}'} \\
\\
ListaDeclVar & \rightarrow & \texttt{IDENTIFICADOR}\ DeclVar\ \texttt{':'}\ Tipo\ \texttt{';'}\ ListaDeclVar \\
 & & \mid \texttt{IDENTIFICADOR}\ DeclVar\ \texttt{':'}\ Tipo\ \texttt{';'} \\
\\
DeclVar & \rightarrow & \epsilon \\
 & & \mid \texttt{','}\ \texttt{IDENTIFICADOR}\ DeclVar \\
\\
Tipo & \rightarrow & \texttt{INT} \\
 & & \mid \texttt{CAR} \\
\\
ListaComando & \rightarrow & Comando \\
 & & \mid Comando\ ListaComando \\
\\
Comando & \rightarrow & \texttt{';'} \\
 & & \mid Expr\ \texttt{';'} \\
 & & \mid \texttt{LEIA}\ \texttt{IDENTIFICADOR}\ \texttt{';'} \\
 & & \mid \texttt{ESCREVA}\ Expr\ \texttt{';'} \\
 & & \mid \texttt{ESCREVA}\ \texttt{CADEIACARACTERES}\ \texttt{';'} \\
 & & \mid \texttt{NOVALINHA}\ \texttt{';'} \\
 & & \mid \texttt{SE}\ \texttt{'('}\ Expr\ \texttt{')'}\ \texttt{ENTAO}\ Comando\ \texttt{FIMSE} \\
 & & \mid \texttt{SE}\ \texttt{'('}\ Expr\ \texttt{')'}\ \texttt{ENTAO}\ Comando\ \texttt{SENAO}\ Comando\ \texttt{FIMSE} \\
 & & \mid \texttt{ENQUANTO}\texttt{'('}\ Expr\ \texttt{')'}\ Comando \\
 & & \mid Bloco \\
\\
Expr & \rightarrow & OrExpr \\
 & & \mid \texttt{IDENTIFICADOR}\ \texttt{'='}\ Expr
\end{array}
$$

$$
\begin{array}{lcl}
OrExpr & \rightarrow & OrExpr\ \texttt{OU}\ AndExpr \\
 & & \mid AndExpr \\
 & & \\
AndExpr & \rightarrow & AndExpr\ \texttt{E}\ EqExpr \\
 & & \mid EqExpr \\
 & & \\
EqExpr & \rightarrow & EqExpr\ \texttt{IGUAL}\ DesigExpr \\
 & & \mid EqExpr\ \texttt{DIFERENTE}\ DesigExpr \\
 & & \mid DesigExpr \\
 & & \\
DesigExpr & \rightarrow & DesigExpr\ \texttt{'<'}\ AddExpr \\
 & & \mid DesigExpr\ \texttt{'>'}\ AddExpr \\
 & & \mid DesigExpr\ \texttt{MAIORIGUAL}\ AddExpr \\
 & & \mid DesigExpr\ \texttt{MENORIGUAL}\ AddExpr \\
 & & \mid AddExpr \\
 & & \\
AddExpr & \rightarrow & AddExpr\ \texttt{'+'}\ MulExpr \\
 & & \mid AddExpr\ \texttt{'-'}\ MulExpr \\
 & & \mid MulExpr \\
 & & \\
MulExpr & \rightarrow & MulExpr\ \texttt{'*'}\ UnExpr \\
 & & \mid MulExpr\ \texttt{'/'}\ UnExpr \\
 & & \mid UnExpr \\
 & & \\
UnExpr & \rightarrow & \texttt{'-'}\ PrimExpr \\
 & & \mid \texttt{'!'}\ PrimExpr \\
 & & \mid PrimExpr \\
 & & \\
PrimExpr & \rightarrow & \texttt{IDENTIFICADOR} \\
 & & \mid \texttt{CARCONST} \\
 & & \mid \texttt{INTCONST} \\
 & & \mid \texttt{'('}\ Expr\ \texttt{')'}
\end{array}
$$

A seguir são apresentadas as definições de tokens que não correspondem a um único caracter. A primeira coluna do tabela 1 corresponde ao *token* e a segunda coluna corresponde à definição dos lexemas que podem aparecer no *token* correspondente à primeira coluna do quadro. Os alunos devem criar expressões regulares no arquivo de entrada para o Flex de modo que o analisador léxico retorne o token e o lexema correspondente encontrado na entrada.

3 Geração dos analisadores sintático e léxico para a linguagem G-V1

Devem ser criados os arquivos de entrada para o Bison e o Flex. Os arquivos devem ser compilados e ligados em um único executável denominado `g-v1`. No Bloco da disciplina na Plataforma Turing há slides que explicam o uso do Bison e do Flex e há também um diretório compactado contendo um exemplo de como gerar analisadores léxico e sintático para uma

Token	**Definição**
PRINCIPAL	principal
IDENTIFICADOR	Inicia com letra ou `'_'` seguido por zero ou mais: letras, dígitos ou `'_'`.
INT	int
CAR	car
LEIA	leia
ESCREVA	escreva
NOVALINHA	novalinha
SE	se
ENTAO	entao
SENAO	senao
FIMSE	fimse
ENQUANTO	enquanto
CADEIACARACTERES	Inicia e termina com `"` e pode ter qualquer caractere exceto `\n`.
OU	||
E	&
IGUAL	==
DIFERENTE	!=
MAIORIGUAL	`>=`
MENORIGUAL	`<=`
CARCONST	Caractere delimitado por '. Inclui caracteres com escape: `'\n'`, `'\t'`, etc.
INTCONST	Constante dos números inteiros, podendo ou ter sinal (+ ou -) ou não.

Tabela 1: Os tokens que não correspondem a apenas um caractere na linguagem G-V1. A tabela mostra o nome do token na primeira coluna e o padrão de formação dos lexemas na segunda coluna.

linguagem de exemplo, a *SimpleLang*. Na mesma pasta se encontra um exemplo de arquivo Makefile que pode ser adaptado para esse trabalho.

3.1 Analisador Léxico

A função que implementa o analisador léxico deverá ser gerada automaticamente, utilizando-se o Flex. O Flex gera um analisador léxico escrito na linguagem C. O Flex possui, ainda, a opção de gerar um analisador léxico na linguagem C++.

A função gerada (`yylex()`) deve ser capaz de reconhecer os *tokens* da gramática da linguagem G-V1 especificada na Seção 2. Os tokens correspondem aos símbolos que aparecem entre apóstrofes nos lados direitos das regras da gramática e também aos símbolos que aparecem escritos em letras maiúsculas na gramática. No último caso, a tabela 1 descreve para cada um desses tokens as palavras neles contidas.

No caso em que o *token* é uma `CADEIACARACTERES` ou `IDENTIFICADOR`, `CARCONT` ou `INTCONST`, a função `yylex()` deve obter como lexema, o texto que forma a *string* ou o identificador (ex.: `x1`, `cont2`) retornando-o no ponteiro `yytext` para o analisador sintático.

O analisador léxico deverá também processar e descartar comentários. Os comentários podem ocupar mais de uma linha do programa fonte. Um comentário em G-V1 inicia com o par de símbolos `/*` e termina com o par de símbolos `*/`. Um erro deve ser reportado, caso um comentário não termine.

O analisador léxico deve reportar, através de mensagens, erros léxicos encontrados no arquivo de entrada. São exemplos de erros léxicos: caracteres inválidos na linguagem, comentários que não terminam, cadeia de caracteres que não terminam ou que ocupam mais de uma linha no arquivo de entrada.

Caso o arquivo contenha um erro léxico, o programa deverá imprimir uma linha contendo exatamente o seguinte: **ERRO:**, seguido por um espaço em branco e, por uma das seguintes mensagens de erro: **CARACTERE INVÁLIDO** ou **COMENTÁRIO NAO TERMINA** ou **CADEIA DE CARACTERES OCUPA MAIS DE UMA LINHA**. Após a mensagem de erro o programa deve imprimir, na mesma linha, o número da linha do programa fonte onde o erro foi encontrado.

Geralmente, o código obtido por uma ferramenta geradora de analisador léxico corresponde a uma função na linguagem para a qual o analisador é gerado. Por exemplo, o Flex e o Lex geram uma função denominada `yylex()`. essa função é do tipo `int` e precisa retornar ao programa que a chama uma constante inteira que indica o token encontrado. A função deve retornar -1 (EOF) quando o fim de arquivo (padrão `<<EOF>>`, veja manual da ferramenta) é encontrado. A função `yylex()` armazena em uma variável global `yytext` o lexema encontrado na entrada. Essa variável é um ponteiro para `char` e deve ser uma variável global, pois seu conteúdo será acessado pelo código gerado pela ferramenta e pelo programa que irá chamar a função `yytext()`.

Material de apoio

- Slides da aula sobre o Flex disponíveis no Bloco 1 da Plataforma Turing.
- Gnu Flex manual
- Pasta compactada com o exemplo de entradas para o Bison e Flex disponível no Bloco 2 da Plataforma Turing.

3.2 Analisador Sintático

A função do compilador que implementa o analisador sintático pode ser gerada automaticamente utilizando-se um gerador de analisadores sintáticos ou *parsers*. Neste trabalho será utilizado o gerador Bison. O Bison utiliza o método LALR para a geração do analisador sintático. Estudaremos esse método e como o Bison consegue gerar automaticamente o analisador sintático a partir da gramática de entrada na segunda etapa da disciplina. O trabalho de implementação do analisador sintático consiste na preparação do arquivo de entrada para o gerador de analisador sintático e na adaptação da função analisador sintático gerada (`yyparse()`) para que possa funcionar utilizando a função analisador léxico gerada pelo Lex/-Flex (`yylex()`).

Deve ser implementado o corpo da função yyerror(), de tal modo que erros sintáticos detectados pela função `yyparse()` sejam emitidos na tela do computador, juntamente com o número da linha onde o erro foi detectado. A mensagem de erro deve iniciar com a palavra **ERRO:** seguida por um espaço.

É importante lembrar que os analisadores léxico e sintáticos são programas separados gerados na linguagem C. Eles precisam ser compilados separadamente e ligados para formarem um único programa. Porém, eles definem variáveis globais nos seus códigos. Para que um modulo m_2 em C possa usar uma variável ou função global definida em outro módulo m_1, m_2 precisa avisar ao compilador e ao ligador de código (*linkeditor* que o objeto a ser utilizado foi definido externamente, no módulo m_1. Para isso, no módulo m_2 a declaração do objeto a ser utilizado deve ser repetida, mas precedida pela palavra `extern`. As váriáveis `yylineno` e `yytext` e a função `yylex()` são declaradas no arquivo em linguagem C gerado pelo Flex. Logo, o arquivo de entrada para o Bisson precisa ter as seguintes declarações:

- `extern int yylineno;`
- `extern char* yytext;`
- `extern int yylex();`

A variável global `yylineno;` contém a linha do arquivo de entrada onde o token/lexema foi encontrado. A função `yylex()` é a função na linguagem C que corresponde ao analisador léxico produzido pelo ferramenta geradora (Lex, Flex). Essa função também é definida globalmente no arquivo em C gerado pela ferramenta geradora de analisador léxico utilizada. Logo, ela precisa ser re-declarada no arquivo de entrada para o Bison precedida por `extern`.

Os objetos acima são utilizados no arquivo em linguagem C gerado pelo pelo Bison. Essa declarações devem ocorrer na primeira seção do arquivo de entrada, entre %{ e %}.

A função `int main(int argc, char** argv)` pode ser declarada ao final do arquivo de entrada para o Bison/Yacc, ou em um arquivo separado. O nome do arquivo de entrada deve ser passado como parâmetro para o argumento `argv[1]` da função `main()`. A função `main()` deve abrir esse arquivo usando o ponteiro de arquivo definido como `extern FILE *yyin;` (pois `yyin` é declarado no código gerado pelo Flex). Dessa forma, dado que o programa executável do analisador sintático tenha o nome *g-v1*, e supondo que o arquivo de entrada seja *teste.g*, deve ser possível executar a análise sintática do arquivo através do seguinte comando em uma *shell* do Linux: `./g-v1 teste.g`. A variável global `yyin` é utilizada pela função `yylex()` gerada pela ferramenta para processar o arquivo de entrada, lendo caractere por caractere do mesmo e formando tokens. A função `main()` deve abrir o arquivo com o

nome em `argv[1]}` armazenando o ponteiro para o arquivo aberto em `yyin` (descrito acima). Ao término da análise sintática, o arquivo passado como parâmetro deve ser fechado.

Material de apoio

- Slides da aula sobre o Bison disponíveis no Bloco 1 da Plataforma Turing.
- Gnu Bison Manual
- Pasta compactada com o exemplo de entradas para o Bison e Flex disponível no Bloco 2 da Plataforma Turing.

4 Geração da representação intermediária do programa de entrada

A representação intermediária corresponde a uma árvore sintática abstrata (*abstract sintax tree* - AST) que é gerada durante a fase de análise sintática (ver Seção 2.8 e 5.3 do livro-texto [1]). Ações semânticas devem ser acrescentadas às produções da gramática no arquivo de entrada para o Bison, formando assim, o esquema de tradução necessário para gerar a árvore abstrata para programas de entrada escritos em G-V1. A árvore abstrata criada deve ser armazenada na memória principal do computador e servirá de entrada para as fases seguintes que correspondem à análise semântica e à geração de código em *assembly*.

É importante que a árvore abstrata seja a mais compacta possível, pois durante a fases seguintes será necessário fazer percursos na árvore para detectar erros semânticos e gerar o código em linguagem *assembly*. Os itens léxicos (identificadores, nomes de comandos, operadores, etc) devem ser armazenados na árvore juntamente com a linha onde eles ocorrem no programa fonte, para que as mensagens de erro possam se referir à linha onde o erro foi detectado.

Material de apoio

- Slides da aula sobre esquema de tradução disponíveis no Bloco 2 da Plataforma Turing.
- O exemplo de construção de árvore sintática abstrata apresentado na Seção 2.8 do livro-texto para a linguagem SimpleLang.
- Slides e pelos códigos fornecidos pelo professor (disponíveis no Bloco 2 na Turing) para o referido exemplo.
- Capítulo 6 do livro *Introduction to Compilers and Language Desing*[2]

5 Tabela de Símbolos para a linguagem G-V1

Conforme mostra a descrição da gramática, um bloco pode conter uma lista de declaração de variáveis locais (que pode ser vazia) e pode ou não ter uma lista de comandos. Porém, um

[1] Alfred Aho et al. Compiladores - Princípios, Técnicas e Ferramentas. segunda edição, Pearson Addison Wesley, São Paulo, 2007.

[2] https://dthain.github.io/books/compiler/

comando da lista de comandos pode ser um outro bloco. Ou seja, os blocos aninham-se uns dentro dos outros. Além disso, cada bloco que possui uma declaração de variáveis locais inicia um escopo novo. Essa situação permite a coexistência de variáveis distintas que possuem o mesmo lexema de identificador (mesmo nome), por exemplo um mesmo nome x declarado em um escopo mais externo e outro objeto de nome x declarado em um escopo mais interno. Veja explicações mais detalhadas no slide sobre tabelas de símbolos na Plataforma Turing.

Nesta parte do trabalho deve ser implementada uma pilha de tabela de símbolos (ou tabela de lexemas de identificadores) capaz de armazenar lexemas de identificador. Essa estrutura deve ter associada a ela o seguinte conjunto de operações:

a) iniciar a pilha de tabela de símbolos - operação que cria uma estrutura de dados inicialmente vazia para representar a pilha de tabelas de símbolos (pilha de escopos);

b) criar uma nova tabela de símbolos (novo escopo) e empilhá-la na pilha de tabela de símbolos.

c) pesquisar por um nome (lexema de identificador) na pilha de tabelas de símbolos. Essa operação deve iniciar a pesquisa na tabela que está no topo da pilha (tabela de símbolos correspondente ao último escopo criado) e enquanto não encontrar, procurar nas tabelas seguintes, do topo em direção à base da pilha de tabelas de símbolos. Deve retornar um ponteiro para a entrada da tabela de símbolos onde o nome foi encontrado ou um ponteiro vazio, caso o nome procurado não se encontre na pilha de tabelas.

d) remover escopo atual - essa função elimina a tabela de símbolos que está no topo da pilha de tabelas.

Material de apoio

- Slides da aula sobre tabelas de símblos disponíveis no Bloco 2 da Plataforma Turing.
- Capítulo 7 do livro *Introduction to Compilers and Language Desing*[3]

6 Analisador Semântico

Aspectos Semânticos da Linguagem G-V1

Declaração de variáveis

As regras de escopo e de tempo de vida das variáveis, da linguagem G-V1 são semelhantes às regras da linguagem C, porém há as seguintes restrições:

- Não há variáveis globais declaradas antes do programa principal, como ocorre na linguagem C (Haverá na versão G-V2 da linguagem).
- Não há o conceito de variáveis `static` como na linguagem C. O tempo de vida de uma variável local a um bloco corresponde à duração do bloco.
- Todo programa em G-V1 ocupa apenas um arquivo e não há compilação separada. Portanto, não há declarações do tipo `extern` como na linguagem C.

[3] https://dthain.github.io/books/compiler/

- Uma variável declarada em um bloco é local a esse bloco e global a todos os blocos internos a esse bloco, em qualquer profundidade de aninhamento de blocos. Entretanto, se uma variável local a um bloco tem o mesmo nome de uma variável global a esse bloco, a variável local se sobrepõe à variável global durante a duração do bloco.

A declaração de qualquer nome deve preceder o seu uso em um escopo válido, isto é, um nome pode ser utilizado em um comando somente se ele tiver sido declarado previamente e se o comando estiver no escopo de sua declaração.

Tipos e Checagem de Tipos

A linguagem G-V1 possui apenas dois tipos distintos: int e car[4]. A linguagem não permite operações entre objetos ou expressões de tipo simples (car ou int) distintos.

A expressão do lado direito de uma atribuição deve ter o mesmo tipo da variável que recebe a expressão. Os operadores aritméticos devem ser aplicados em expressões do tipo `int`. Os operadores relacionais devem ser aplicados a operandos de mesmo tipo.

Não existe o tipo lógico explícito na linguagem G-V1, entretanto nos comandos `se` e `enquanto` é necessário avaliar se uma expressão é verdadeira ou falsa para a tomada de uma decisão quanto ao fluxo de execução dos comandos internos a estes dois comandos. Assim como a linguagem C, a linguagem G-V1 trata como verdadeira uma expressão cujo valor é diferente de zero e como falsa, uma expressão cujo valor é zero. Obviamente, essa avaliação é possível somente em tempo de execução do programa fonte. Entretanto, a nível de análise semântica, uma expressão lógica (com operações OR, AND ou NEGAÇÃO) ou relacional (com operadores relacionais – >, <,<=, etc) devem receber tipo `int`.

O analisador semântico deve percorrer a árvore abstrata gerada para uma cadeia de entrada (programa escrito em G-V1) para realizar a análise semântica. Um programa sintaticamente correto corresponde a um ou mais blocos. Um bloco possui zero ou mais declarações de variáveis seguida por uma lista de comandos. Um comando pode ser outro bloco. Essa estrutura hierárquica da linguagem faz com que na árvore abstrata, em um bloco mais externo, as declarações de variáveis apareçam na subárvore à esquerda da subárvore correspondente à lista de comando do bloco. Se nesse bloco, (subárvore direita) ocore em algum momento outro bloco, na subárvore a ele correspondente, as declarações de variáveis desse bloco interno ficam em uma subárvore esquerda e a lista de comando fica na subárvore à direita. Portanto, o controle de escopo e toda a análise semântica podem ser realizados através de um percurso que se inicia na raiz da AST em direção às folhas e da esquerda para a direita.

No percurso em uma árvore/subárvore correspondente a um bloco, o analisador encontrará primeiro as declarações de variáveis. Com isso ele pode criar uma nova tabela de símbolos (novo escopo) correspondente à declaração e ir incluindo os identificadores de variávis e seus respectivos tipos na tabela criada. Ao iniciar um percurso numa subávore que corresponde a uma lista de comandos do bloco, os identificadores válidos encontrados nos comando devem estar em uma das tabelas de símbolos da pilha de tabelas de símbolos. No topo, estará a tabela de símbolos das últimas declaracções visitadas na AST e, na base, os identificadores possívelmente declarados no bloco mais externo da função `principal`.

Ao terminar um percurso em uma árvore/subárvore correspondente a um bloco, a tablea de símbolos do topo da pilha (escopo atual) é removida. Esse percurso permite checar, gradualmente, a correção semântica de cada construção presente na árvore, seguindo as regras

[4]Na versão 2 da linguagem, haverá o tipo vetor (array) de int e de car.

semânticas apresentadas nesta seção. Qualquer ocorrência de uma construção no programa fonte que não é permitida na linguagem G-V1 deve ser considerada um erro semântico. Nesse caso, o analisador semântico deve reportar o erro semântico encontrado, o número da linha onde ele ocorreu e terminar a compilação.

Não há um material de apoio específico para esta fase. O(s) aluno(s) devem recorrer aos seus conhecimentos de Estrutura de Dados para realizar o percurso na árvore e ir manipulando a pilha de tabela de símbolos, à medida que o percurso ocorre.

7 Geração de código

Se o programa de entrada estiver semanticamente correto, outro algoritmo de percurso deve ser iniciado na AST. Desta vez, o percurso será com o objetivo de gerar código. Nesse novo percurso, tabelas de símbolos devem ser criadas e empilhadas na pilha de tabelas à medida que o percurso encontra subárvores de declarações de variáveis. Também como ocorre durante o percurso de análise semântica, uma tabela é eliminada do topo da pillha ao final de uma subárvore correspondente a um bloco, se houve pelo menos uma declaração de variáveis nesse bloco. Como o programa está semanticamente correto, as informações na tabela não servem para checagem de escopo e tipo, mas sim, para identificar as posições de cada variável na memória e também o tamanho do espaço a ser alocado para cada variável (indicado pelo seu tipo).

Material de apoio

- Slides da aula sobre geração de código, disponíveis no Bloco 2 da Plataforma Turing.
- MIPS Assembly Language Programming (pdf disponível na Platarfoma Turing).

8 Informações Sobre a Implementação e a Entrega

O trabalho pode ser desenvolvido em grupo de dois alunos e deve ser entregue até o dia 07/05/2026 via tarefa criada no site da disciplina na plataforma Turing. O trabalho deve ser apresentado durante as aulas dos dias 27/04/206, 30/04/2026 e 04/05/2026 pela turma de Ciência da Computação e nos dias 30/04/2026, 05/05/2026 e 07/05/2026 pela turma de Engenharia de Computaçao. Os trabalho são totalmente factíveis individualmente. Portanto, se um dos componentes abandonar o grupo, o outro deve continuar os trabalho individualmente e apresentá-lo nas datas marcadas.

Deve ser entregue uma pasta compactada contendo:

- O arquivo de especificações para o gerador de analisador léxico utilizado (arquivo de entrada para o Flex/Lex - g-v1.l).
- O arquivo de especificações para o gerador de analisador sintático expandido com ações semânticas necessárias para a geração de árvore sintática abstrata (arquivo de entrada para o Yacc/Bison - g-v1.y).
- O código em C/C++ que implementa a tabela de símbolos e o arquivo headerfile correspondente.

- O código em C/C++ do analisador semântico que irá utilizar a árvore abstrata e a tabela de símbolos para detectar a ocorrência de erros semânticos em arquivos contendo programas escritos na linguagem Goianinha (e headerfile correspondente).
- O código me C/C++ que irá utilizar a árvore abstrata e a tabela de símbolos para gerar o código objeto em linguagem *assembly*.
- Um arquivo Makefile com comandos para:
  - Gerar o código do analisador léxico a partir do arquivo contendo as especificações para o gerador de analisador léxico.
  - Gerar o código do analisador sintático e construtor da árvore abstrata a partir do arquivo contendo as especificações para o gerador de analisador sintático.
  - Compilar os arquivos gerados e os escritos na linguagem de programação adotada no projeto (C/C++).
  - Gerar o executável (O professor deve ser capaz de obter o executável do trabalho digitando apenas o comando *make* em uma *shell* do Linux).

A pasta compactada deve ser submetida como uma tarefa a ser criada no site da disciplina. Os códigos gerados devem estar preparados para executarem no sistema operacional Linux (ambiente onde os trabalhos serão avaliados). O compilador C/C++ utilizado pelo professor para compilar os códigos será o gcc/g++ na versão 14. O Flex utilizado será o de **versão 2.6.4**. O Bison utilizado será o de **versão 3.8.2**.

Sugestão de Cronograma

Para que o grupo consiga concluir o trabalho no prazo previsto (27/04/2026), sugere-se o seguinte cronograma, que atribui dias a cada fase de implementação do trabalho, de acordo com a complexidade da fase:

Tarefa	Data Final	Num. Dias
Analisadores léxicos e sintáticos integrados	21/03/2026	8
Geração da AST	31/03/2026	10
Implementação da tabela de símbolos	11/04/2026	11
Análisador semântico	18/04/2026	7
Geração de código	26/04/2026	8

Tabela 2: Cronograma sugerido.

Importante:

Cópias idênticas ou modificadas dos códigos ou de partes dos códigos são consideradas plágios. **Plágio é crime**. O aluno que cometer plágio em seu trabalho receberá nota zero no mesmo. **SE O COMPILADOR GERADO ESTIVER CORRETO, MAS O ALUNO NÃO O APRESENTAR OU NÃO SOUBER EXPLICAR O TRABALHO, RECEBERÁ NOTA ZERO NO MESMO**.